

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 81 01-1-ЛУ

Исполнители:

Студент группы БДРИП241

_____ / С. В. Гаврилин /
«__» _____ 2026 г.

Студент группы БДРИП242

_____ / А. А. Клушин /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.03-04 81 01-1-ЛУ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Пояснительная записка

RU.17701729.09.03-04 81 01-1

Листов 28

Москва 2026

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Пояснительная записка – это документ, содержащий описание целей и задач проекта, обоснование принятых технических решений, алгоритмов функционирования программы и организации входных и выходных данных, в соответствии с которым производится оценка качества разработки программы.

Настоящая Пояснительная записка к курсовому проекту «Платформа для no-code автоматизации бизнес-процессов» содержит следующие разделы: «Введение», «Назначение разработки», «Технические характеристики», «Технико-экономические показатели», «Ожидаемые качественные и количественные показатели», приложения [8].

В разделе «Введение» указано наименование программы и краткая характеристика области применения.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

Раздел «Технические характеристики» содержит постановку задачи на разработку, анализ девяти продуктов-аналогов, описание и обоснование выбора архитектуры, технологического стека, ключевых алгоритмов (исполнение workflow-графа, изоляция пользовательского Python-кода, версионирование), подробное описание схемы реляционной базы данных и REST API серверной части, а также описание метода организации входных и выходных данных.

Раздел «Технико-экономические показатели» содержит информацию об ориентировочной экономической эффективности разработки.

Раздел «Ожидаемые качественные и количественные показатели» содержит перечень целевых числовых метрик системы: время отклика, производительность, объём автоматических тестов, показатели покрытия.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.103-77 [1]: Обозначения программ и программных документов.
2. ГОСТ 19.104-78 [2]: Основные надписи.
3. ГОСТ 19.105-78 [3]: Общие требования к программным документам.
4. ГОСТ 19.106-78 [4]: Требования к программным документам, выполненным печатным способом.
5. ГОСТ 19.404-79 [8]: Пояснительная записка. Требования к содержанию и оформлению.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.09.03-04 81 01-1

Изменения к данной Пояснительной записке оформляются согласно ГОСТ 19.603-78, ГОСТ 19.604-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	6
1.1. Наименование программы	6
1.2. Краткая характеристика области применения	6
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	7
2.1. Функциональное назначение	7
2.2. Эксплуатационное назначение	8
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	9
3.1. Постановка задачи на разработку программы	9
3.1.1. Анализ продуктов-аналогов	11
3.2. Описание алгоритма и функционирования программы	13
3.2.1. Архитектурный подход	13
3.2.2. Технологический стек	13
3.2.3. Алгоритм исполнения workflow-графа	14
3.2.4. Изолированное исполнение пользовательского кода	15
3.2.5. Версионирование workflow и снапшоты	16
3.2.6. Визуальный редактор workflow	16
3.2.7. Трансляция событий по WebSocket	17
3.2.8. Контент-адресуемое хранение конфигураций	17
3.2.9. Схема реляционной базы данных и миграции	18
3.2.10. Программный интерфейс REST API	19
3.2.11. Стратегия автоматического тестирования и непрерывной интеграции	19
3.3. Описание и обоснование выбора метода организации входных и выходных данных ...	20
3.3.1. Описание метода организации входных и выходных данных	20
3.3.2. Обоснование выбора метода	20
3.4. Требования к составу и параметрам технических средств	21
4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	22
4.1. Ориентировочная экономическая эффективность	22
4.2. Предполагаемая потребность	22
4.3. Экономические преимущества по сравнению с аналогами	22
5. ОЖИДАЕМЫЕ КАЧЕСТВЕННЫЕ И КОЛИЧЕСТВЕННЫЕ ПОКАЗАТЕЛИ	23

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
ПРИЛОЖЕНИЕ. ТЕРМИНОЛОГИЯ	26

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование темы разработки — «Платформа для no-code автоматизации бизнес-процессов».

Наименование темы разработки на английском языке — «No-code business process automation platform».

Условное обозначение — «FluxPilot».

1.2. Краткая характеристика области применения

Программный продукт «Платформа для no-code автоматизации бизнес-процессов» представляет собой веб-платформу, предназначенную для визуального проектирования и автоматического исполнения бизнес-процессов (workflow) без необходимости написания программного кода со стороны конечного пользователя. Платформа предоставляет интерактивный редактор, позволяющий конструировать workflow из набора готовых типов узлов и связывать их направленными рёбрами в виде направленного ациклического графа. Готовый workflow допускает запуск вручную, по расписанию на основе стоп-выражения и по входящему webhook-запросу.

Программный продукт применим для решения следующих классов задач: интеграция между внутренними и внешними информационными системами через HTTP-запросы; периодическая выгрузка, преобразование и загрузка данных по сценарию ETL; реактивная обработка событий из внешних источников; проведение А/В-экспериментов с разделением пользовательского трафика по стратегии и статистической проверкой значимости разности конверсий; автоматизация рутинных операций аналитиков и продуктовых менеджеров. Движок исполнения workflow по концепции совместим с существующими решениями (в частности, продуктом n8n), однако расширен механизмами двухуровневого версионирования с возможностью отката, именованных снапшотов уровня документа, контент-адресуемого хранения конфигураций узлов с дедупликацией, изолированного исполнения пользовательского кода в одноразовых Docker-контейнерах и серверного модуля продуктовой аналитики со статистическим тестированием.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Программный продукт предназначен для автоматизации бизнес-процессов без необходимости написания программного кода конечным пользователем. Программный продукт состоит из двух функциональных частей. Серверная часть реализована на языке программирования Kotlin версии 2.2.21 с использованием каркаса Spring Boot версии 3.5.3, исполняемого на платформе JDK 24; серверная часть обслуживает REST API и WebSocket-канал, поддерживает миграции схемы PostgreSQL средствами Liquibase, использует объектное хранилище MinIO для конфигураций узлов, применяет систему контейнеризации Docker для изолированного исполнения пользовательского кода и предоставляет фоновый планировщик cron-задач. Клиентская часть реализована на языке программирования TypeScript версии 5.6 с использованием каркаса Angular версии 19.2; клиентская часть предоставляет интерактивный визуальный редактор workflow с холстом drag-and-drop, палитрой типов узлов, инспектором конфигурации, панелями истории запусков, подробного просмотра одного запуска, именованных снапшотов и продуктовой аналитики, а также онбординг-тур для новых пользователей.

Программный продукт обеспечивает выполнение четырнадцати функциональных требований, перечисленных в общем техническом задании (RU.17701729.09.03-04 ТЗ 01-1, раздел 4.1.1): визуальный редактор workflow; система триггеров трёх типов (ручной, scheduler по cron-выражению, webhook); HTTP-узел с конфигурируемыми методами, заголовками и тайм-аутом; узлы обработки потоков данных (фильтрация, преобразование, агрегация, обход с побочным эффектом, разворачивание вложенных списков); узлы Code Node с Python- и JavaScript-песочницами; параллельное исполнение независимых ветвей с обработкой сбоев; двухуровневое версионирование (автоматические ревизии и именованные версии); интерфейс просмотра состояния исполнения (входные и выходные данные узлов, ошибки, история запусков); feature-флаги для управления функциональными блоками без повторного развёртывания; WebSocket-трансляция событий исполнения в режиме реального времени; контент-адресуемое хранение конфигураций узлов в MinIO с дедупликацией; A/B-аналитика workflow с узлами разделения и слияния ветвей и серверным модулем статистического тестирования; именованные снапшоты workflow уровня документа; режим пошаговой отладки workflow с возможностью запуска одной выбранной ноды с произвольным входным значением без полного прогона графа.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2.2. Эксплуатационное назначение

Программный продукт предназначен для эксплуатации тремя основными группами пользователей. Первая группа — бизнес-аналитики и продуктовые менеджеры, использующие платформу для автоматизации повторяющихся задач интеграции данных между информационными системами организации и для проведения А/В-экспериментов над пользовательским трафиком. Указанная группа пользователей формирует основную целевую аудиторию платформы и описана в исследовательских артефактах проектной документации в форме связанного JTBD-сценария. Вторая группа — системные администраторы и DevOps-инженеры, разворачивающие платформу в инфраструктуре организации через Docker Compose или манифесты оркестратора Kubernetes и конфигурирующие интеграции с PostgreSQL, MinIO и системами мониторинга. Третья группа — разработчики, использующие платформу как gateway-инструмент для построения сложных конвейеров обработки данных и интеграции с внутренними прикладными интерфейсами организации.

Программный продукт развёрнут в продакшен-окружении и публично доступен по адресу <https://fluxpilot.ru/>. Развёртывание реализовано в соответствии с конфигурацией `deploy/docker-compose.prod.yml` и включает контейнер PostgreSQL 16, контейнер MinIO, контейнер серверной части на Spring Boot, контейнер клиентской части на Nginx с TLS-сертификатом Let's Encrypt и контейнер `docker-socket-proxy`. Интерактивная документация REST API доступна по адресу <https://fluxpilot.ru/v1/swagger-ui.html>.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Целью проекта является разработка собственного программного движка исполнения workflow с поддержкой отказоустойчивости, параллельного исполнения и горизонтального масштабирования, совместимого по концепции с продуктом n8n (модель узлов и связей), но расширенного механизмами версионирования с откатом, именованных снапшотов, контент-адресуемого хранения с дедупликацией, изолированного исполнения пользовательского кода и серверного модуля продуктовой аналитики. Проект выполняется в составе командной курсовой работы из двух студентов.

Задачи проекта декомпозированы на следующие функциональные компоненты. Реализация визуального редактора workflow с интерактивным холстом SVG, drag-and-drop размещением узлов, рисованием рёбер кривыми Безье, режимами зума и панорамирования, селекцией узлов, инспектором конфигурации с динамическим выбором формы по типу узла, панелями истории запусков, подробного просмотра одного запуска, именованных снапшотов и продуктовой аналитики А/В-эксперимента. Реализация ядра исполнения workflow на серверной стороне с десериализацией графа из jsonb-поля реляционной базы данных, валидацией ацикличности через топологическую сортировку, построением event-driven цепочки исполнения через интерфейс CompletableFuture в пуле потоков, изоляцией сбоев и трансляцией событий по WebSocket-каналу. Реализация системы триггеров трёх типов с фоновым планировщиком stop-задач и приёмом webhook-запросов. Реализация исполнителей узлов десяти типов (HTTP, пять операций обработки потоков данных, Code Node для Python и JavaScript, узлы разделения и слияния ветвей А/В-эксперимента, внутренний webhook-исполнитель). Реализация изолированного исполнения пользовательского кода в одноразовых Docker-контейнерах с системными ограничениями. Реализация двухуровневой схемы версионирования и независимого механизма именованных снапшотов уровня документа. Реализация контент-адресуемого хранения конфигураций узлов в MinIO с дедупликацией по криптографическому хешу SHA-256. Реализация интерфейса мониторинга исполнения через WebSocket-канал с подсветкой узлов на холсте редактора в реальном времени.

Распределение задач между участниками проектной группы приведено в таблице 1.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Участник	Зона ответственности	Конкретные задачи
Гаврилин С.В. (БДРИП241)	Клиентская часть, пользовательский опыт, end-to-end-тестирование	Визуальный редактор workflow на Angular: интерактивный холст с drag-and-drop, рисование рёбер SVG-кривыми Безье, zoom и pan, селекция узлов; компоненты палитры узлов, инспектора конфигурации, боковой панели запусков, подробного просмотра одного запуска, панели именованных снапшотов, вкладки продуктовой A/B-аналитики и универсального модального контейнера; онбординг-тур; адаптивная вёрстка с пятью брейкпойнтами; REST- и WebSocket-фасады в директориях core/api/ и core/ws/; сервисы уровня приложения; набор end-to-end-сценариев на Playwright. Документально подтверждается индивидуальной частью документации с шифрами с суффиксом «02-1».
Клушин А.А. (БДРИП242)	Серверная часть, инфраструктура, автоматизация тестирования и развёртывания	Движок исполнения на Kotlin с топологической сортировкой по алгоритму Кана и event-driven исполнением через CompletableFuture; исполнители узлов всех десяти типов; контент-адресуемое хранение конфигураций в MinIO; двухуровневое версионирование и именованные снапшоты; WebSocket-брокер; серверный модуль A/B-аналитики со статистическим тестом; миграции Liquibase; конфигурация Spring Boot; модульные и интеграционные тесты на JUnit и Testcontainers; CI/CD-пайплайн на GitHub Actions; продакшен-развёртывание через Docker Compose. Документально подтверждается индивидуальной частью документации с шифрами с суффиксом «03-1».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Таблица 1. Распределение задач между участниками проектной группы

3.1.1. Анализ продуктов-аналогов

В целях обоснования архитектурных и продуктовых решений был проведён анализ девяти существующих программных продуктов в области автоматизации бизнес-процессов и оркестрации workflow. Анализ преследовал три цели: определение функциональных возможностей, являющихся отраслевым стандартом и подлежащих обязательной реализации; выявление функциональных пробелов существующих решений, которые разрабатываемая платформа может закрыть; обоснование архитектурных и продуктовых решений проекта со ссылкой на удачные и неудачные подходы продуктов-аналогов.

Продукт n8n представляет собой open-source платформу автоматизации workflow с визуальным редактором на базе веб-интерфейса. Продукт поддерживает значительное количество готовых интеграций с внешними сервисами; запуск workflow возможен вручную, по расписанию и по входящему webhook-запросу. Слабыми сторонами продукта являются отсутствие встроенного механизма версионирования workflow (пользователь работает только с текущей версией; откат к ранее сохранённому состоянию возможен только через внешний Git-репозиторий с экспортом JSON-файлов), ограниченная поддержка параллельного исполнения ветвей и недостаточная изоляция пользовательского кода (продукт использует deprecated-библиотеку VM2 с известными уязвимостями). Разрабатываемая платформа повторяет n8n в части концепции узлов и связей с визуальным редактором, однако устраняет указанные пробелы.

Продукт Zapier представляет собой облачный SaaS-сервис автоматизации, ориентированный на широкую аудиторию бизнес-пользователей без технической подготовки. Сильной стороной продукта является обширная библиотека готовых коннекторов. Слабыми сторонами являются проприетарный характер сервиса без возможности самостоятельного развёртывания, отсутствие параллельного исполнения ветвей и отсутствие возможности исполнения произвольного пользовательского кода. Для классов организаций, требующих self-hosted развёртывания (банковский сектор, государственные структуры, оборонная промышленность), Zapier неприменим.

Продукт Make (ранее Integromat) представляет собой визуальный конструктор автоматизаций с поддержкой параллельных маршрутов и условного ветвления через router-модули. Продукт является проприетарным SaaS-сервисом. Слабыми сторонами являются отсутствие версионирования workflow за пределами undo/redo текущей сессии редактирования, ограниченные

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

возможности исполнения пользовательского кода и ценовая модель, затрудняющая планирование расходов при значительных объёмах запусков.

Продукт Apache Airflow представляет собой open-source платформу оркестрации данных, широко применяемую для построения ETL- и ELT-конвейеров. Workflow в Apache Airflow описывается программно на языке Python в виде DAG, что обеспечивает выразительность и интеграцию с системой контроля версий Git. Продукт поддерживает параллельное исполнение и масштабирование на множестве worker-узлов. Слабыми сторонами являются отсутствие визуального редактора, высокий порог входа и тяжёлая инфраструктура. Разрабатываемая платформа дополняет нишу Apache Airflow визуальным интерфейсом для непрограммистов при сохранении production-ready движка исполнения.

Продукт Node-RED представляет собой open-source инструмент визуального программирования потоков. Слабыми сторонами являются однопоточный движок, что ограничивает применимость продукта к CPU-интенсивным операциям, отсутствие версионирования и отсутствие изолированной песочницы для пользовательского кода.

Продукт Prefect представляет собой open-source платформу оркестрации, позиционируемую как современная альтернатива Apache Airflow. Workflow описываются Python-декораторами. Слабой стороной является отсутствие визуального редактора.

Продукт Temporal представляет собой open-source платформу оркестрации долгоживущих workflow с гарантией исполнения по схеме durable execution. Слабой стороной является отсутствие визуального редактора и значительный объём адаптации мышления, требуемый концепцией activity/workflow.

Продукт Microsoft Power Automate представляет собой облачную low-code и no-code платформу компании Microsoft. Слабыми сторонами являются проприетарный характер с привязкой к экосистеме поставщика и отсутствие возможности self-hosted развёртывания.

Продукт Camunda представляет собой open-source платформу оркестрации на базе стандарта BPMN 2.0. Слабой стороной является избыточность нотации BPMN для простых сценариев автоматизации и значительный порог входа в концепции стандарта.

Разрабатываемый программный продукт занимает нишу между лёгкими SaaS-конструкторами и тяжёлыми платформами оркестрации. Продукт предоставляет визуальный редактор для непрограммистов и одновременно реализует production-ready движок исполнения на платформе JVM с поддержкой параллелизма, изолированной Docker-песочницы, двухуровневого

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

версионирования с откатом и контент-адресуемого хранения с дедупликацией. Дополнительной отличительной характеристикой является встроенная поддержка А/В-экспериментов как функциональности первого уровня, отсутствующая во всех рассмотренных продуктах-аналогах.

3.2. Описание алгоритма и функционирования программы

В настоящем разделе приведены описания основных архитектурных и алгоритмических решений программного продукта. Подробные технические детали реализации серверной и клиентской частей вынесены в индивидуальные пояснительные записки участников проектной группы (RU.17701729.09.03-04 81 03-1 для серверной части, RU.17701729.09.03-04 81 02-1 для клиентской части).

3.2.1. Архитектурный подход

Серверная часть программного продукта реализована в виде модульного монолита: одно Spring Boot приложение, упакованное в исполняемый JAR-файл, с внутренним разделением исходного кода на изолированные пакеты с однонаправленными зависимостями. Клиентская часть реализована в виде одностраничного приложения на каркасе Angular с организацией исходного кода по принципу feature folders и разделением на pages, components, services и core. На уровне системы в целом применяется двухкомпонентная архитектура «backend + frontend» с фиксированным программным контрактом через REST API и WebSocket-канал.

Альтернативные варианты архитектуры — единый одиночный монолит без модулей и микросервисная архитектура — были отвергнуты. Одиночный монолит без модулей ведёт к быстрому росту неявных зависимостей при увеличении кодовой базы и затрудняет рефакторинг. Микросервисная архитектура требует значительного объёма инфраструктурной обвязки при незначительном выигрыше на стадии MVP курсовой работы. Модульный монолит предоставляет компромисс между указанными подходами, обеспечивая дисциплину границ модулей при простоте развёртывания. Подробное описание состава пакетов серверной части приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.1).

3.2.2. Технологический стек

Серверная часть реализована на языке Kotlin версии 2.2.21 с использованием каркаса Spring Boot версии 3.5.3 на платформе JDK 24. Клиентская часть реализована на языке TypeScript версии 5.6 с использованием каркаса Angular версии 19.2 с библиотекой реактивных потоков RxJS, библиотекой построения диаграмм Chart.js, библиотекой Angular CDK для drag-and-drop,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

библиотеками `@stomp/stompjs` и `sockjs-client` для WebSocket-клиента. Для хранения структурированных данных применяется реляционная база данных PostgreSQL версии 16 с миграциями схемы средствами Liquibase. Для хранения объёмных конфигураций узлов применяется объектное хранилище MinIO с контент-адресуемой схемой. Для трансляции событий применяется протокол WebSocket с прикладным протоколом STOMP поверх SockJS-транспорта. Для автоматической документации REST API применяется библиотека `springdoc-openapi` версии 2.7.0. Для системы непрерывной интеграции применяется платформа GitHub Actions.

Выбор языка серверной части обусловлен сочетанием строгой статической типизации с поддержкой `null-safety`, эффективной модели многопоточности через пулы потоков JVM, развитой экосистемой готовых модулей каркаса Spring и компактностью синтаксиса Kotlin по сравнению с Java. Альтернативные варианты языка серверной части (Go, Node.js, Python) были признаны менее подходящими, подробное обоснование приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.2). Выбор языка клиентской части обусловлен встроенной структурированностью каркаса Angular, готовыми механизмами Dependency Injection и маршрутизации, наличием библиотеки Angular CDK для построения интерактивных интерфейсов и TypeScript-first подходом разработки.

3.2.3. Алгоритм исполнения workflow-графа

Workflow в программном продукте представляется направленным ациклическим графом, вершины которого соответствуют узлам обработки, а рёбра — связям передачи данных. Ядро исполнения применяет комбинированный алгоритм: на этапе подготовки выполняется топологическая сортировка графа по алгоритму Кана (одновременно служащая проверкой ацикличности), после чего строится event-driven цепочка экземпляров `CompletableFuture` и выполняется параллельное исполнение в пуле потоков фиксированного размера. Альтернативные подходы — последовательный обход в глубину или ширину и уровневое исполнение после топологической сортировки — были отвергнуты по причинам отсутствия параллелизма и наличия искусственной синхронизации на границах уровней.

Реализация ядра представлена классом `WorkflowExecutionService` в пакете `ru.startem.aelevena.run` серверной части. Алгоритм включает следующие основные шаги: чтение записи запуска и связанной ревизии из реляционной базы данных, десериализацию графа в доменные объекты, проверку ацикличности и вычисление топологического порядка по алгоритму Кана, создание записей результатов узлов со статусом «`queued`», построение цепочки

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

CompletableFuture с обработчиками thenApplyAsync в пуле потоков, обработку сбоев через обработчик whenComplete с транзитивным распространением статуса «skipped» по потомкам сбойного узла, определение итогового статуса запуска и трансляцию событий жизненного цикла в WebSocket-канал. Корректность алгоритма подтверждается набором интеграционных тестов с реальными контейнерами PostgreSQL и MinIO. Подробное описание алгоритма приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.3).

Помимо штатного исполнения workflow целиком ядро поддерживает режим пошаговой отладки: пользователь может запустить одну выбранную ноду графа с произвольным входным значением, не прогоняя предшествующие узлы. Указанный режим использует ту же реестровую инфраструктуру исполнителей и тот же механизм трансляции событий, что и штатное исполнение; результат возвращается клиентской части в синхронном виде и одновременно фиксируется в истории запусков с пометкой отладочного исполнения. Режим предназначен для итеративной разработки конфигурации отдельных узлов (выражений фильтрации, фрагментов пользовательского кода, шаблонов HTTP-запросов).

В качестве направлений дальнейшего развития ядра предусмотрено горизонтальное масштабирование исполнения на множество реплик серверной части с использованием реляционной базы данных PostgreSQL в качестве распределённой очереди задач (claim записей через SELECT ... FOR UPDATE SKIP LOCKED, lease-механизм с heartbeat для перезапуска незавершённых runs при отказе реплики, advisory lock для выбора лидера cron-планировщика, механизм LISTEN/NOTIFY для трансляции WebSocket-событий между репликами) без введения дополнительных инфраструктурных сервисов.

3.2.4. Изолированное исполнение пользовательского кода

Узел Code Node предоставляет пользователю возможность исполнения произвольного программного кода на Python или JavaScript в составе workflow. Серверная часть реализует изоляцию пользовательского кода через запуск одноразового Docker-контейнера с системными ограничениями. Альтернативные подходы (исполнение в JVM через GraalVM Polyglot, дочерний процесс с применением системных механизмов rlimit и seccomp) были отвергнуты по соображениям недостаточной изоляции и сложности конфигурации.

Реализация представлена классами PythonNodeExecutor и JavaScriptNodeExecutor, делегирующими запуск контейнера общему универсальному движку ContainerSandboxRunner. Базовые образы — python:3.11-slim для Python и node:20-alpine для JavaScript. На старте сер-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

верной части вспомогательный класс `SandboxImageWarmer` выполняет предзагрузку базовых образов. Команда запуска контейнера применяет следующие флаги изоляции: `--rm -i --network none --read-only --tmpfs /tmp:rw,size=16m --cap-drop ALL --security-opt no-new-privileges --memory 256m --cpus 1 --pids-limit 64`. Тайм-аут исполнения пользовательского кода составляет 5 секунд плюс служебный overhead 30 секунд на запуск контейнера и инициализацию интерпретатора. Подробное описание алгоритма приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.4).

3.2.5. Версионирование workflow и снапшоты

Программный продукт поддерживает двухуровневое версионирование workflow и независимый механизм именованных снапшотов уровня документа. Первый уровень — автоматические ревизии: каждое сохранение конфигурации создаёт новую запись в таблице `workflow_revision` с инкрементальным номером и полным сериализованным графом. Второй уровень — именованные версии: по явному запросу пользователя создаётся запись в таблице `workflow_version` со ссылкой на конкретную ревизию через внешний ключ с ограничением `ON DELETE RESTRICT`. Третий механизм — именованные снапшоты уровня документа в таблице `workflow_snapshot` с пользовательскими полями имени и описания, предназначенные для фиксации значимых вех развития workflow. Альтернативные подходы (хранение различий между ревизиями, event sourcing) были отвергнуты по соображениям сложности реализации и замедления чтения. Подробное описание схемы хранения приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.5).

Клиентская часть предоставляет панель управления именованными снапшотами и интерфейс переключения между ревизиями. Сохранение конфигурации workflow в редакторе реализовано с применением механизма `debounced-save`: после каждого изменения через 1 секунду неактивности автоматически отправляется запрос обновления, создающий новую ревизию на серверной стороне.

3.2.6. Визуальный редактор workflow

Визуальный редактор workflow является основным интерфейсом программного продукта. Редактор обеспечивает интуитивное размещение узлов через `drag-and-drop` из палитры на холст, создание связей между узлами через перетаскивание от выходного порта к входному, перемещение существующих узлов с автоматическим обновлением подключённых связей, масштабирование холста колесом мыши с центрированием в позиции курсора, панорамирование

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

путём перетаскивания пустого пространства, селекцию и групповую селекцию узлов, копирование и удаление узлов, динамическую панель инспектора с подгрузкой формы конфигурации в зависимости от типа выделенного узла.

Главный компонент редактора — `WorkflowCanvasComponent`. Каждый узел представляет собой отдельный Angular-компонент `WorkflowNodeComponent` с абсолютным позиционированием через CSS-свойство `transform`. Рёбра реализованы SVG path-элементами внутри единого корневого SVG. Перетаскивание узлов из палитры реализовано через библиотеку Angular CDK с кастомным обработчиком `drop`. Селекция реализована через механизм Angular Signals. Инспектор подгружает форму через `switch` по типу выбранного узла. Адаптивная вёрстка реализована через `@media-queries` в файле `styles.css` с пятью брейкпойнтами (480, 640, 900, 1200, 1920 пикселей). Подробное описание реализации приведено в индивидуальной пояснительной записке студента Гаврилина С.В.

3.2.7. Трансляция событий по WebSocket

Программный продукт обеспечивает трансляцию событий жизненного цикла исполнения от серверной части клиентской в режиме реального времени с использованием протокола WebSocket с прикладным протоколом STOMP поверх SockJS-транспорта. Альтернативные подходы — Server-Sent Events и long polling — были отвергнуты по соображениям отсутствия поддержки топиков-подписок и высокой латентности соответственно.

Серверная сторона реализована классом `WebSocketConfig` и слушателем `GraphBroadcastListener`, преобразующим доменные события изменения состояния узлов в STOMP-сообщения с публикацией в топик `/topic/workflows/{workflowId}/graph`. Клиентская сторона реализована сервисом `WorkflowWsService`, использующим библиотеки `@stomp/stompjs` и `sockjs-client` с настройкой переподключения и heartbeat. Поток обновлений транслируется в Angular Signals компонента редактора для реактивного обновления визуального состояния узлов.

3.2.8. Контент-адресуемое хранение конфигураций

Программный продукт реализует контент-адресуемое хранение конфигураций узлов в объектном хранилище MinIO с дедупликацией по криптографическому хешу SHA-256. Сохранение конфигурации включает приведение JSON-дерева к канонической форме (утилита `CanonicalJson`), вычисление SHA-256-хеша (утилита `Hashing`), запись объекта в MinIO по ключу `blobs/{hash}` с регистрацией в индексной таблице `blob_index`. Указанная схема обеспечивает однократное хранение содержательно идентичных конфигураций между ревизиями и версиями

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

workflow. Подробное описание схемы приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.6).

3.2.9. Схема реляционной базы данных и миграции

Схема реляционной базы данных управляется библиотекой Liquibase с описанием changeset-ов в формате YAML. Альтернативная библиотека Flyway была отвергнута в пользу Liquibase по причине поддержки YAML-формата с улучшенной читаемостью, наличия богатого набора preCondition и rollback-сценариев. Реализовано пять инкрементальных changeset-ов: 001-init.yaml (базовый набор таблиц с внешними ключами, индексами, правилами каскадного удаления и ON DELETE RESTRICT для исторических версий), 002-trigger-node-id.yaml (привязка триггера к стартовому узлу), 003-restore-after-drop.yaml (защита от непреднамеренного DROP в dev-окружении), 004-add-is-demo.yaml (флаг is_demo для посевных workflow), 005-workflow-snapshots.yaml (таблица workflow_snapshot для именованных снапшотов с описанием). Состав основных таблиц схемы приведён в таблице 2.

Таблица	Назначение	Ключевые поля
workflows	Центральная сущность workflow	id (UUID PK), name, description, current_version_id (FK), is_demo
workflow_revision	Автоматические ревизии конфигурации workflow	id (PK), workflow_id (FK ON DELETE CASCADE), revision_number, graph_skeleton (jsonb), created_at
workflow_version	Именованные версии уровня ревизии	id (PK), workflow_id (FK), root_revision_id (FK ON DELETE RESTRICT), version_tag
workflow_snapshot	Именованные снапшоты уровня документа	id (PK), workflow_id (FK), revision_id (FK), name, description
workflow_run	Запуски workflow	id (PK), workflow_id (FK), workflow_revision_id (FK), status, trigger_type, started_at, finished_at, input, output
node_run	Результат исполнения каждого узла	id (PK), workflow_run_id (FK ON DELETE CASCADE), node_id, node_type,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Таблица	Назначение	Ключевые поля
		status, config_hash, input_json, output_json, error_message
triggers	Триггеры запуска workflow	id (PK), workflow_id (FK), type (enum), config (jsonb), token (UNIQUE NULL), enabled, node_id
blob_index	Индекс контент-адресуемого хранилища	content_hash (PK, SHA-256), s3_key, size_bytes, created_at

Таблица 2. Основные таблицы базы данных

3.2.10. Программный интерфейс REST API

Клиентская часть программного продукта и внешние информационные системы взаимодействуют с серверной частью через программный интерфейс REST. Документация интерфейса генерируется автоматически библиотекой springdoc-openapi с публикацией спецификации OpenAPI 3.0 и интерактивного интерфейса Swagger UI по адресу /v1/swagger-ui.html. Серверная часть содержит шесть REST-контроллеров (WorkflowsController, WorkflowVersionsController, WorkflowSnapshotsController, RunsController, TriggersController, AbAnalyticsController) и реализует двадцать один REST-эндпойнт. Обработка ошибок выполняется глобальным обработчиком RestExceptionHandler с единообразным форматом ответа, содержащим машинно-читаемый код ошибки и человеко-читаемое сообщение. Подробное описание программного интерфейса приведено в индивидуальной пояснительной записке студента Клушина А.А. (раздел 3.2.12).

3.2.11. Стратегия автоматического тестирования и непрерывной интеграции

Программный продукт сопровождается многоуровневой стратегией автоматического тестирования. Серверная часть тестируется модульными тестами на JUnit Jupiter и интеграционными тестами на Spring Boot Test с применением библиотеки Testcontainers для запуска реальных контейнеров PostgreSQL и MinIO. Клиентская часть тестируется модульными тестами на Jasmine с применением Karma в качестве test runner и end-to-end-тестами на Playwright. Покрытие исходного кода серверной части по строкам, измеряемое библиотекой JaCoCo, составляет не менее 90%; покрытие критических пакетов (run, executor, blob) — не менее 95%.

Пайплайн непрерывной интеграции реализован в платформе GitHub Actions. На каждый pull request и push в основную ветку поднимаются сервисные контейнеры PostgreSQL и MinIO, последовательно выполняются прогон модульных и интеграционных тестов серверной

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

части, прогон модульных тестов клиентской части и прогон end-to-end-тестов. Дополнительные workflow реализуют автоматическое обновление SVG-бейджа покрытия, сборку и публикацию Docker-образов серверной и клиентской частей в GitHub Container Registry, развёртывание на сервер через SSH-action.

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Описание метода организации входных и выходных данных

Входные данные программного продукта поступают из четырёх источников. Первый источник — клиентская часть, передающая пользовательские запросы на CRUD-операции с workflow, запуск исполнения, управление триггерами и снапшотами в виде JSON-тел HTTP-запросов через REST API. Второй источник — внешние информационные системы, передающие произвольное JSON-тело через webhook-эндпойнт в качестве входа запускаемого workflow. Третий источник — внутренний планировщик стоп-задач, инициирующий запуск scheduler-триггеров без внешних входных данных. Четвёртый источник — внешние сервисы, опрашиваемые HTTP-узлами при исполнении workflow, тела ответов которых становятся входами последующих узлов.

Метаданные сущностей хранятся в реляционной базе данных PostgreSQL в типизированных колонках. Структура графа workflow хранится в колонках типа jsonb. Объёмные конфигурации узлов хранятся в объектном хранилище MinIO по контент-адресуемой схеме. Выходные данные формируются в трёх каналах: REST API возвращает JSON-ответы клиентской части и внешним системам, WebSocket-канал транслирует события исполнения клиентской части в режиме реального времени, результаты исполнения сохраняются во внутреннем хранилище в таблицах node_run и workflow_run.

3.3.2. Обоснование выбора метода

Гибридная схема хранения, сочетающая реляционную базу данных PostgreSQL для метаданных и структуры графа в формате jsonb с объектным хранилищем MinIO для объёмных конфигураций узлов в контент-адресуемой схеме, оптимальна для специфики программного продукта. Метаданные сущностей требуют гарантий ACID и индексирования, обеспечиваемых реляционной базой данных. Объёмный содержательный контент требует эффективного дедуплицированного хранилища, обеспечиваемого объектным хранилищем с контент-адресуемой схемой. Канал WebSocket для трансляции событий исключает необходимость периодического опроса со стороны клиентской части. Программный интерфейс REST в качестве основного средства

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

взаимодействия соответствует индустриальному стандарту и обеспечивает совместимость с произвольными HTTP-клиентами.

3.4. Требования к составу и параметрам технических средств

Минимальные требования к техническим средствам для развёртывания серверной части: процессор не менее 4 ядер архитектуры x86_64; оперативная память не менее 8 ГБ; дисковое пространство не менее 20 ГБ. Программные средства серверной части: JDK 24, PostgreSQL 16, MinIO стабильной версии, Docker Engine 20.10 или совместимый. Все программные компоненты допускают развёртывание в Docker-контейнерах через предоставленный файл `deploy/docker-compose.prod.yml`. Технические средства клиентской части: современный браузер с поддержкой стандарта ECMAScript 2020 и протокола WebSocket (Google Chrome 90 и выше, Mozilla Firefox 90 и выше, Apple Safari 15 и выше, Microsoft Edge 90 и выше); разрешение экрана не менее 1024×768 пикселей; стабильное подключение к сети Интернет.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Курсовой проект имеет учебную природу и не предусматривает прямого расчёта экономической эффективности по нормативным методикам. Косвенная оценка прикладной эффективности программного продукта при применении в реальной организации может быть выполнена через следующие соображения. Платформа позволяет бизнес-аналитикам и продуктовым менеджерам организации реализовать значительный объём типовых задач интеграции и автоматизации без привлечения команды разработки. При типовой стоимости часа разработчика 4 000–8 000 рублей и экономии 10–20 часов разработки в месяц на одну автоматизированную задачу экономия одного активного пользователя платформы составляет от 40 000 до 160 000 рублей в месяц. С учётом стоимости развёртывания (один сервер не выше 50 000 рублей в месяц) и сопровождения окупаемость достигается при 1–2 активных пользователях.

4.2. Предполагаемая потребность

Программный продукт востребован организациями, нуждающимися в автоматизации бизнес-процессов и интеграций без привлечения команды разработки. Целевые сегменты включают малый и средний бизнес (стартапы на стадии product-market fit, активно проводящие А/В-эксперименты), крупные организации с продуктовыми и аналитическими командами, организации с требованиями к self-hosted развёртыванию (банковский сектор, государственные структуры, оборонная промышленность), не имеющие возможности использовать облачные SaaS-сервисы.

4.3. Экономические преимущества по сравнению с аналогами

Основные преимущества разрабатываемого программного продукта по сравнению с продуктами-аналогами включают: открытый исходный код с возможностью самостоятельного развёртывания; визуальный редактор для непрограммистов; production-ready движок с поддержкой параллелизма; встроенное двухуровневое версионирование с откатом и независимый механизм именованных снапшотов уровня документа; изолированная Docker-песочница для пользовательского кода; встроенная поддержка А/В-экспериментов со статистическим тестированием в качестве функциональности первого уровня.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. ОЖИДАЕМЫЕ КАЧЕСТВЕННЫЕ И КОЛИЧЕСТВЕННЫЕ ПОКАЗАТЕЛИ

Целевые качественные и количественные показатели программного продукта приведены ниже. Время отклика веб-интерфейса при типовых операциях не превышает 2 секунд. Время от приёма триггера до перехода первого узла workflow в статус «running» не превышает 5 секунд. 95-й перцентиль времени отклика REST API на запросы чтения при типовой нагрузке не превышает 500 мс. 95-й перцентиль задержки WebSocket-канала в локальной сети не превышает 200 мс. Параллелизм исполнения обеспечивается пулом потоков размера не менее 16.

Покрытие исходного кода серверной части по строкам составляет не менее 90%. Количество автоматических тестов: серверные модульные тесты на JUnit Jupiter — около 55; серверные интеграционные тесты на Testcontainers — около 12; клиентские модульные тесты на Jasmine с применением Karma — около 65; end-to-end-тесты на Playwright — около 95. Доля успешно проходящих тестов на ветке main — 100%. Изоляция пользовательского кода в узлах Code Node обеспечивается одноразовым Docker-контейнером с системными ограничениями. Автоматическая документация REST API представлена спецификацией OpenAPI 3.0 и интерактивным интерфейсом Swagger UI. Адаптивная вёрстка клиентской части обеспечивает корректную работу на разрешениях от 480 до 1920 пикселей по ширине с пятью брейкпойнтами. Время полного прогона пайплайна непрерывной интеграции на pull request не превышает 12 минут. Готовность программного продукта после команды `docker compose up -d` достигается за время не более 60 секунд.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Framework Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/reference/> (дата обращения: 03.12.2025).
2. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (дата обращения: 05.12.2025).
3. Kotlin Language Documentation [Электронный ресурс]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 08.12.2025).
4. Angular Documentation [Электронный ресурс] / Google. — URL: <https://angular.dev/> (дата обращения: 10.12.2025).
5. PostgreSQL 16 Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.12.2025).
6. Liquibase Documentation [Электронный ресурс]. — URL: <https://docs.liquibase.com/> (дата обращения: 15.12.2025).
7. OpenAPI Specification 3.0.3 [Электронный ресурс]. — URL: <https://spec.openapis.org/oas/v3.0.3> (дата обращения: 18.12.2025).
8. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).
9. MinIO Object Storage Documentation [Электронный ресурс]. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.02.2026).
10. Docker Engine Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/> (дата обращения: 18.03.2026).
11. Playwright Documentation [Электронный ресурс] / Microsoft. — URL: <https://playwright.dev/> (дата обращения: 18.04.2026).
12. n8n Documentation [Электронный ресурс]. — URL: <https://docs.n8n.io/> (дата обращения: 12.11.2025).
13. Apache Airflow Documentation [Электронный ресурс]. — URL: <https://airflow.apache.org/docs/> (дата обращения: 15.11.2025).
14. Camunda Platform Documentation [Электронный ресурс]. — URL: <https://docs.camunda.org/> (дата обращения: 18.11.2025).
15. Kahn A. B. Topological sorting of large networks

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

16. Клеппман М. Высоконагруженные приложения // Программирование, масштабирование, поддержка. Питер. – 2018.
17. Ньюмен С. Создание микросервисов. 2-е издание. – Питер, 2023.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ. ТЕРМИНОЛОГИЯ

Термин	Значение
Workflow	Автоматизированная последовательность действий, представленная в виде направленного ациклического графа из узлов и связей.
Узел	Минимальная единица исполнения в составе workflow. Узел характеризуется типом, конфигурацией и набором входных и выходных портов.
Связь	Направленное ребро между двумя узлами, обеспечивающее передачу данных.
Триггер	Механизм инициирования запуска workflow. Поддерживаются типы: ручной, scheduler по cron-выражению, webhook.
Ревизия	Автоматически создаваемая копия конфигурации workflow при каждом сохранении в редакторе.
Снапшот	Именованное состояние workflow, фиксируемое пользователем для возможности явного отката или для отметки значимой вехи развития.
Песочница	Изолированная среда исполнения произвольного пользовательского программного кода.
DAG	Directed Acyclic Graph — направленный ациклический граф.
MinIO	Объектное хранилище с открытым исходным кодом, совместимое с программным интерфейсом Amazon S3.
Контент-адресуемое хранение	Схема хранения, при которой ключом доступа к объекту служит криптографический хеш его содержимого.
WebSocket	Двунаправленный полнодуплексный протокол поверх TCP для обмена сообщениями.
STOMP	Simple Text Oriented Messaging Protocol — текстовый прикладной протокол поверх WebSocket с поддержкой топиков публикации и подписки.
SockJS	JavaScript-библиотека и серверный протокол, обеспечивающие совместимость WebSocket с инфраструктурой, не поддерживающей штатный WebSocket-handshake.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Термин	Значение
Docker	Система контейнеризации приложений на базе механизмов Linux namespace и cgroups.
Liquibase	Программный инструмент управления миграциями схемы реляционной базы данных.
OpenAPI	Спецификация описания программных интерфейсов REST, преемник стандарта Swagger 2.0.
JaCoCo	Программная библиотека измерения покрытия исходного кода для платформы JVM.
Testcontainers	Программная библиотека для запуска одноразовых Docker-контейнеров с инфраструктурными зависимостями в интеграционных тестах.
A/B-тестирование	Метод сравнения двух или более вариантов реализации сценария на различных группах пользователей с целью статистического выбора более эффективного.
JTBD	Jobs To Be Done — методология исследования пользователя через формулирование «работ», которые пользователь намерен выполнить.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]